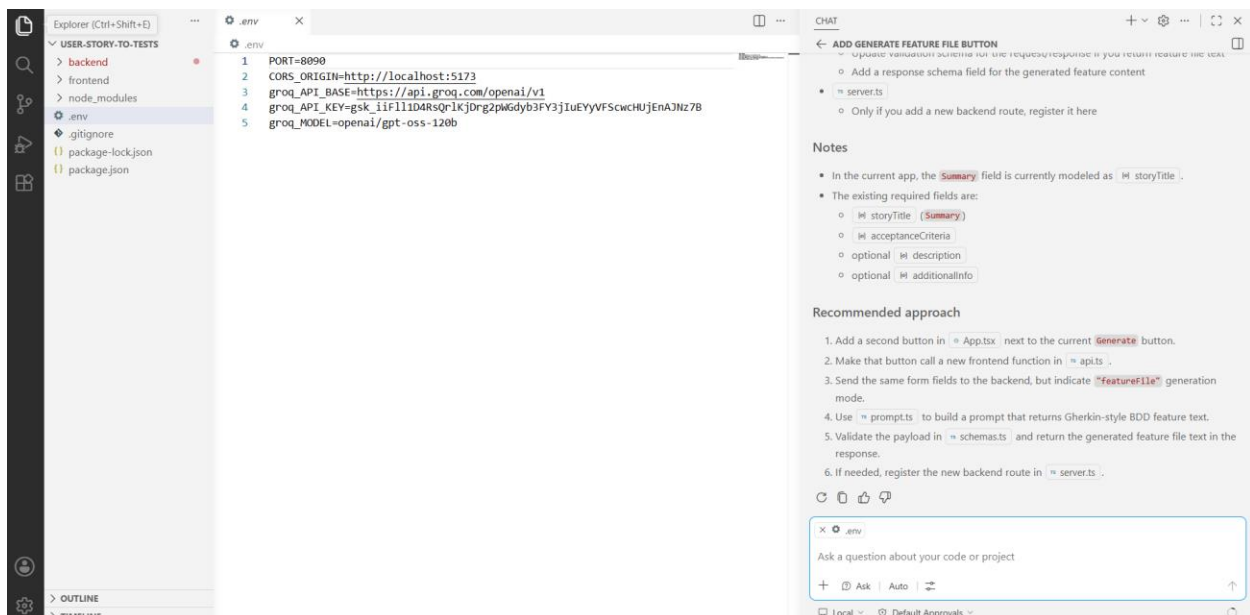
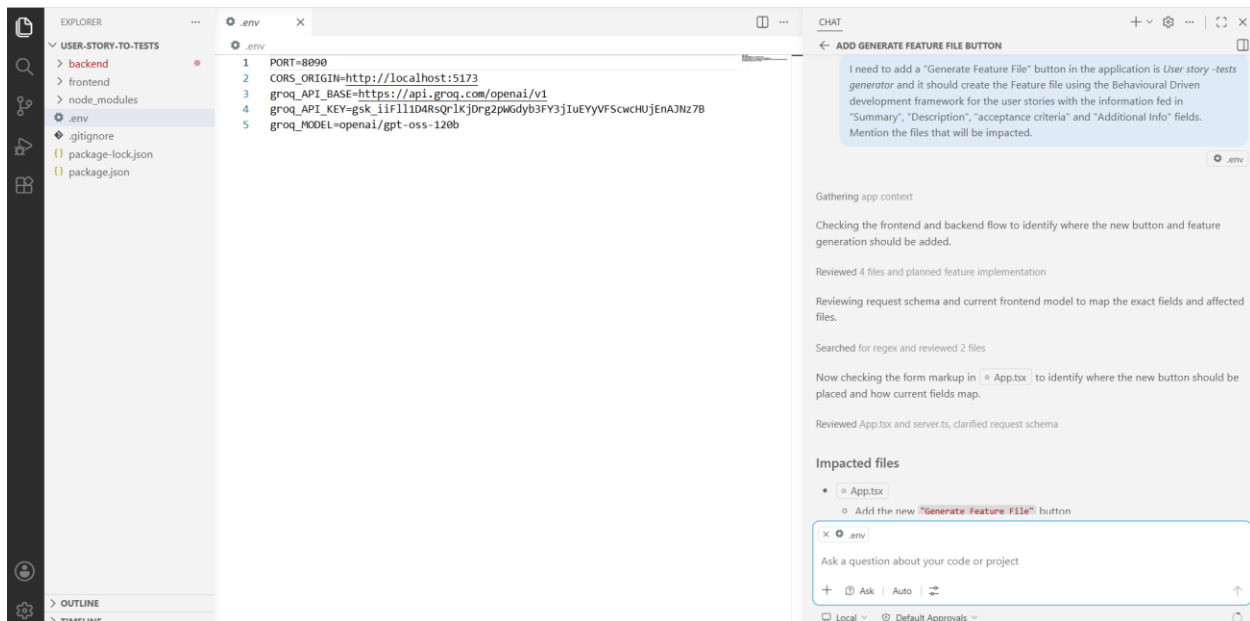


Step 1:

Prompt 1: (In Ask Mode)

I need to add a "Generate Feature File" button in the application is **User story -tests generator** and it should create the Feature file using the Behavioural Driven development framework for the user stories with the information fed in "Summary", "Description", "acceptance criteria" and "Additional Info" fields. Mention the files that will be impacted.



Step 2:

Prompt 2: Implement the recommended approach 1 by 1 by asking questions

EXPLORER

USER-STORY-TO-TESTS

back

src

lim

groqClient.ts

routes

prompts.ts

schemas.ts

server.ts

package.json

tsconfig.json

frontend

node_modules

src

api.ts

App.tsx

main.tsx

types.ts

vite-env.d.ts

env

index.html

package.json

tsconfig.json

tsconfig.node.json

vite.config.ts

node_modules

env

gitignore

package-lock.json

package.json

OUTLINE

TIMELINE

tsconfig.json

App.tsx

api.ts

groqClient.ts

frontend > src > TS api.ts > ...

5 export async function generateTests(request: GenerateRequest): Promise<GenerateResponse> {

19

20

21 const data: GenerateResponse = await response.json()

22 return data

23 } catch (error) {

24 console.error('Error generating tests:', error)

25 throw error instanceof Error ? error : new Error('Unknown error occurred')

26 }

27 }

28

29 export async function generateFeature(request: GenerateRequest): Promise<string> {

30 try {

31 const response = await fetch(`\${API\_BASE\_URL}/generate-feature`, {

32 method: 'POST',

33 headers: {

34 'Content-Type': 'application/json',

35 },

36 body: JSON.stringify(request),

37 })

38

39 if (!response.ok) {

40 const errorData = await response.json().catch(() => ({ error: 'Unknown error'}

41 throw new Error(errorData.error || 'HTTP error! status: \${response.status}')

42 }

43

44 const featureText = await response.text()

45 return featureText

46 } catch (error) {

47 console.error('Error generating feature file:', error)

48 throw error instanceof Error ? error : new Error('Unknown error occurred')

49 }

50 }

CHAT

ADD GENERATE FEATURE FILE BUTTON

Planned implementation details and clarified output preferences

Quick question before I start

1. Should the new `Generate Feature File` button appear next to the existing `Generate` button in the same form?

2. When the feature file is generated, do you want it displayed in the app results area, or downloaded directly as a `.feature` file?

1.yes

Great, I'll place the new `Generate Feature File` button next to the existing `Generate` button.

Next: do you want the generated feature file to be shown in the app results area, or downloaded immediately as a `.feature` file?

both

Implementing step-by-step approach

> 6 files changed +113 -5

Keep Undo

+ TS api.ts

Describe what to build

+ Agent Auto

Local Default Approvals

Ln 27, Col 1 Spaces: 2 UTF-8 LF TypeScript

EXPLORER

USER-STORY-TO-TESTS

back

src

lim

groqClient.ts

routes

prompts.ts

schemas.ts

server.ts

package.json

tsconfig.json

frontend

node_modules

src

api.ts

App.tsx

main.tsx

types.ts

vite-env.d.ts

env

index.html

package.json

tsconfig.json

tsconfig.node.json

vite.config.ts

node_modules

env

gitignore

package-lock.json

package.json

OUTLINE

TIMELINE

tsconfig.json

App.tsx

api.ts

groqClient.ts

frontend > src > TS api.ts > ...

5 export async function generateTests(request: GenerateRequest): Promise<GenerateResponse> {

19

20

21 const data: GenerateResponse = await response.json()

22 return data

23 } catch (error) {

24 console.error('Error generating tests:', error)

25 throw error instanceof Error ? error : new Error('Unknown error occurred')

26 }

27 }

28

29 export async function generateFeature(request: GenerateRequest): Promise<string> {

30 try {

31 const response = await fetch(`\${API\_BASE\_URL}/generate-feature`, {

32 method: 'POST',

33 headers: {

34 'Content-Type': 'application/json',

35 },

36 body: JSON.stringify(request),

37 })

38

39 if (!response.ok) {

40 const errorData = await response.json().catch(() => ({ error: 'Unknown error'}

41 throw new Error(errorData.error || 'HTTP error! status: \${response.status}')

42 }

43

44 const featureText = await response.text()

45 return featureText

46 } catch (error) {

47 console.error('Error generating feature file:', error)

48 throw error instanceof Error ? error : new Error('Unknown error occurred')

49 }

50 }

CHAT

ADD GENERATE FEATURE FILE BUTTON

Planned implementation details and clarified output preferences

/api/generate-feature

Considering backend response

Got it — I'll add `/api/generate-feature` next.

Next decision:

Should the feature endpoint return JSON with a `featureFile` string field in a new response type, or should it return raw text directly?

should it return raw text directly

Implemented step-by-step and inspected backend routes

Inspecting the backend LLM client and preparing the feature route implementation.

Reviewed groqClient.ts and analyzed implementation options

I've got the design details; now applying the frontend and backend updates for raw `.feature` generation and display/download support.

> 6 files changed +113 -5

Keep Undo

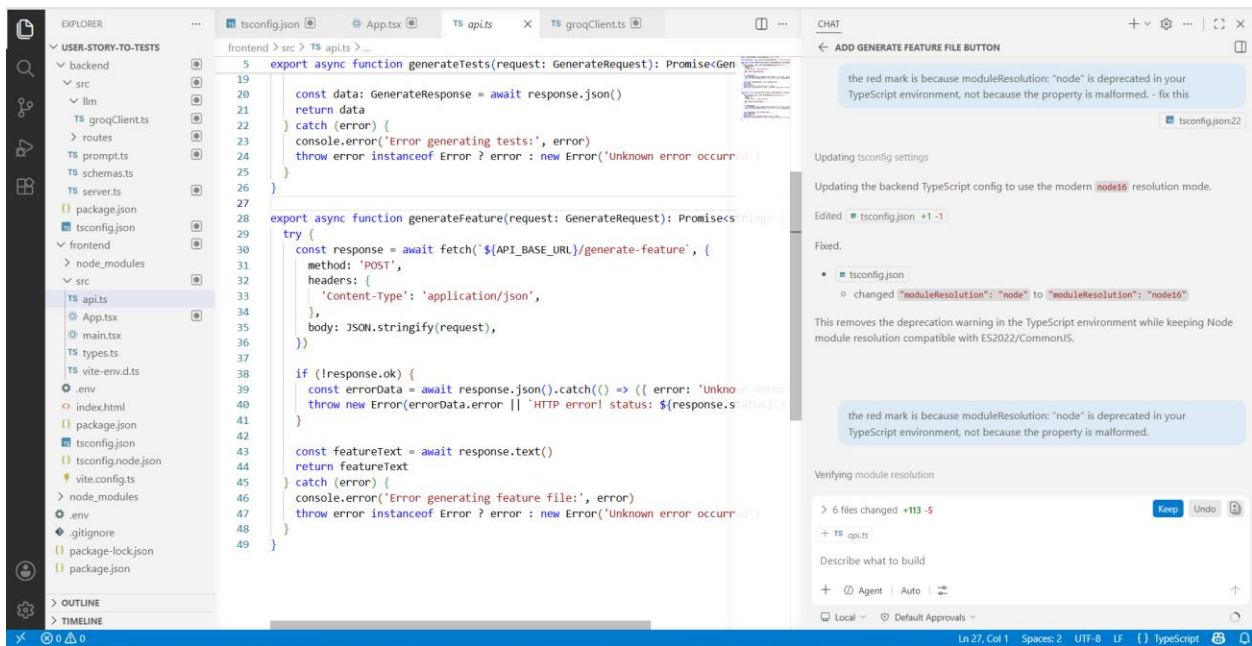
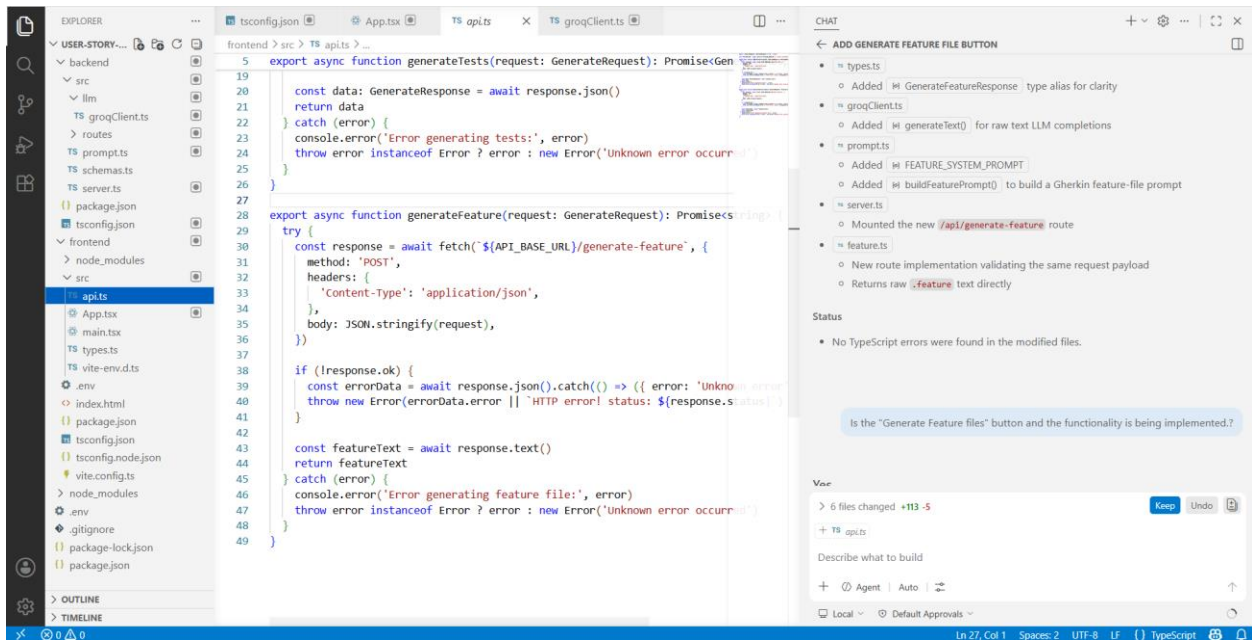
+ TS api.ts

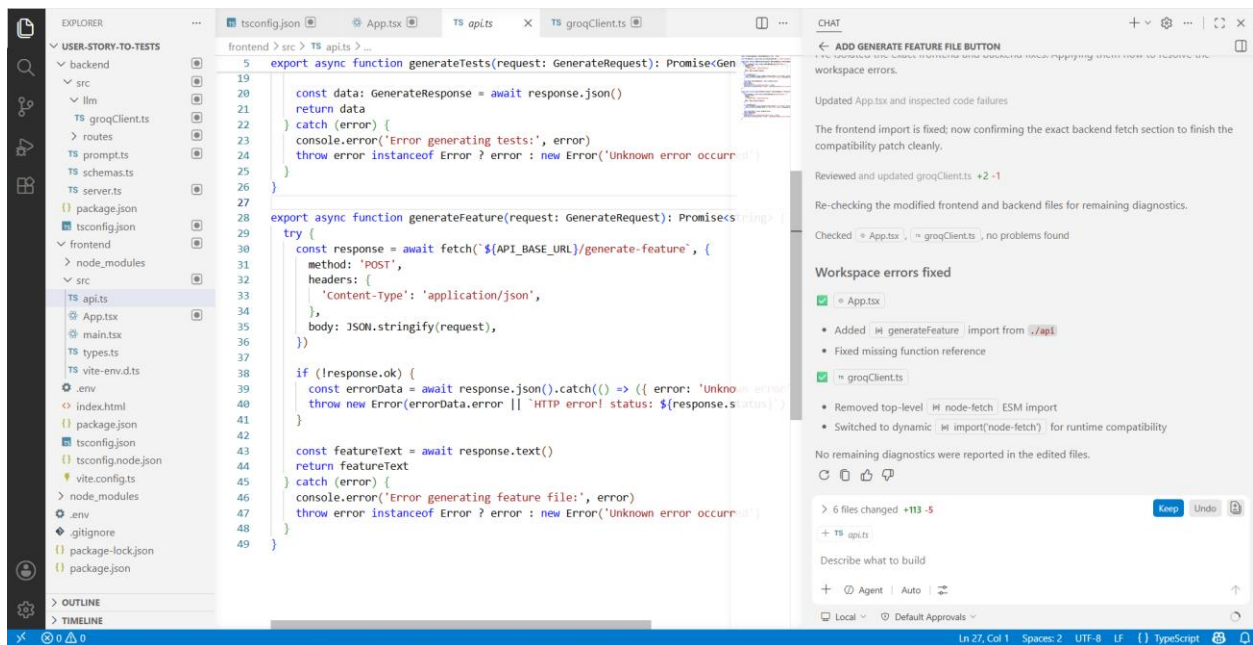
Describe what to build

+ Agent Auto

Local Default Approvals

Ln 27, Col 1 Spaces: 2 UTF-8 LF TypeScript



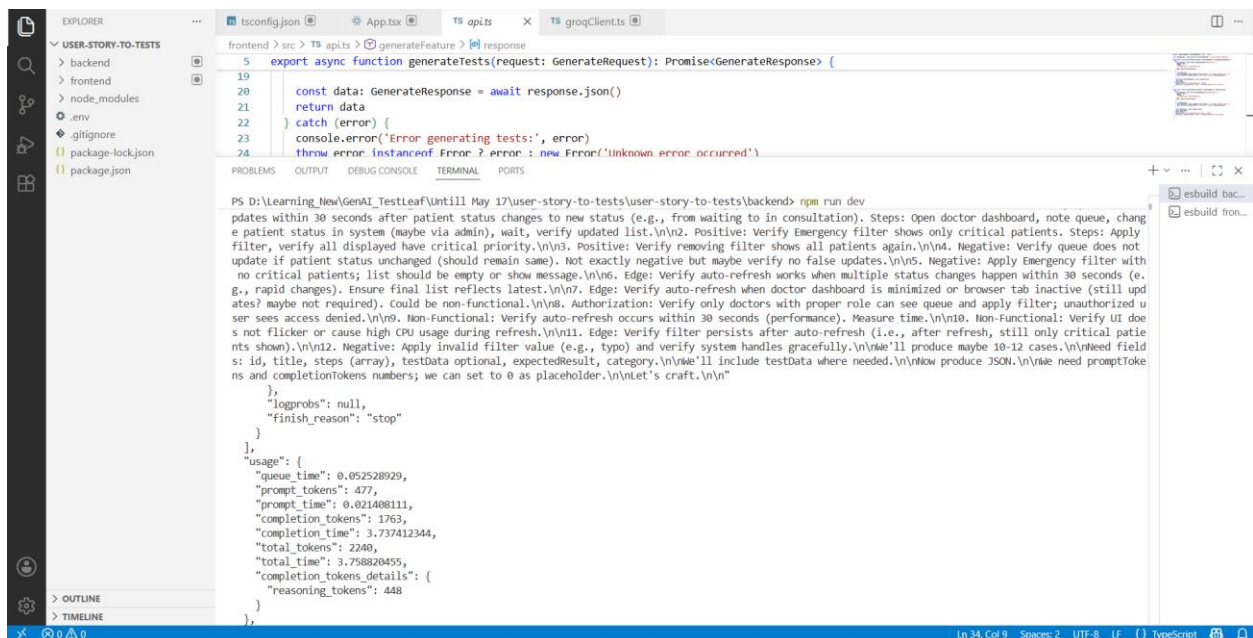


Step 3:

Output:

User story – Test case generator

In Backend Terminal:



localhost:5173

☆

Generate Tests

Generate Feature File

Generated Feature File

Download .feature File

```

Feature: Dynamic Queue List of AHC

  As a ward doctor,
  I want the queue list to refresh automatically and support filtering by patient priority,
  So that I can attend to high-priority patients first.

  Background:
    Given the doctor dashboard is open

  @auto-refresh
  Scenario: Queue list updates automatically when patient status changes
    When a patient status changes in the system
    Then the queue list should update automatically within 30 seconds

  @emergency-filter
  Scenario: View only critical patients using the Emergency filter
    Given the doctor wants to view only critical patients
    When the "Emergency" filter is applied
    Then only patients with critical priority should be visible
    And the queue list should contain no routine patients
          
```

Acceptance Criteria *

Given the doctor dashboard is open, when a patient status changes in the system, then the queue list should update automatically within 30 seconds.

Given the doctor wants to view only critical patients, when the "Emergency" filter is applied, then only patients with critical priority should be visible.

Additional Info

Any additional information (optional)...

Generating...

Generating...

Generating output...

1 Error is identified – “when clicked on the one button both the buttons are getting clicked but the both the functionalities are working fine “Generating....” “Generate Testcases” and “Generate Feature file”

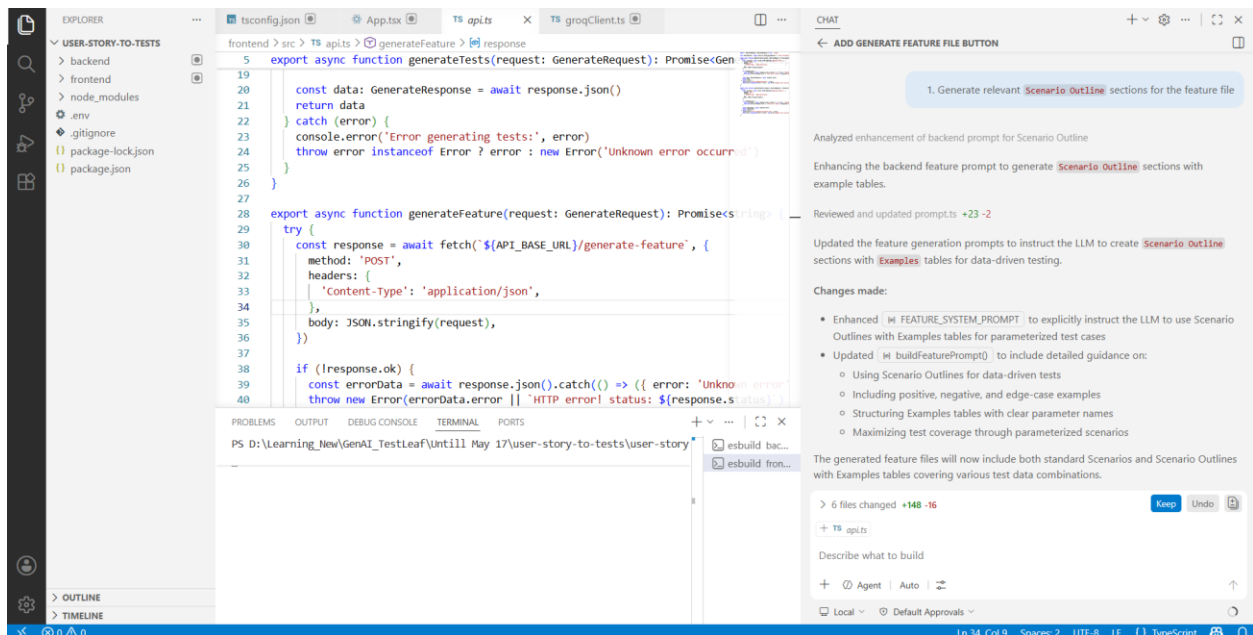
Feature File without Scenario Outline:

```

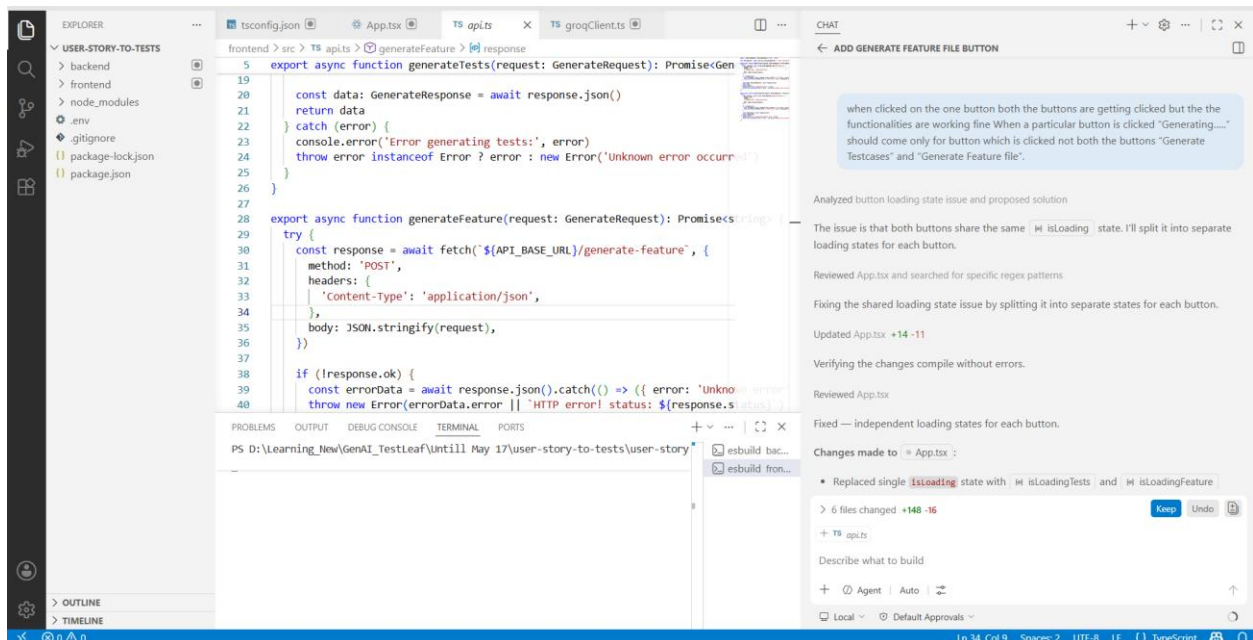
Feature: Dynamic Queue List of AHC
  As a ward doctor,
  I want the queue list to refresh automatically and support filtering by patient priority,
  So that I can attend to high-priority patients first.
  Background:
    Given the doctor dashboard is open
  @auto-refresh
  Scenario: Queue list updates automatically when patient status changes
    When a patient status changes in the system
    Then the queue list should update automatically within 30 seconds
  @emergency-filter
  Scenario: View only critical patients using the Emergency filter
    Given the doctor wants to view only critical patients
    When the "Emergency" filter is applied
    Then only patients with critical priority should be visible
    And the queue list should contain no routine patients
  
```

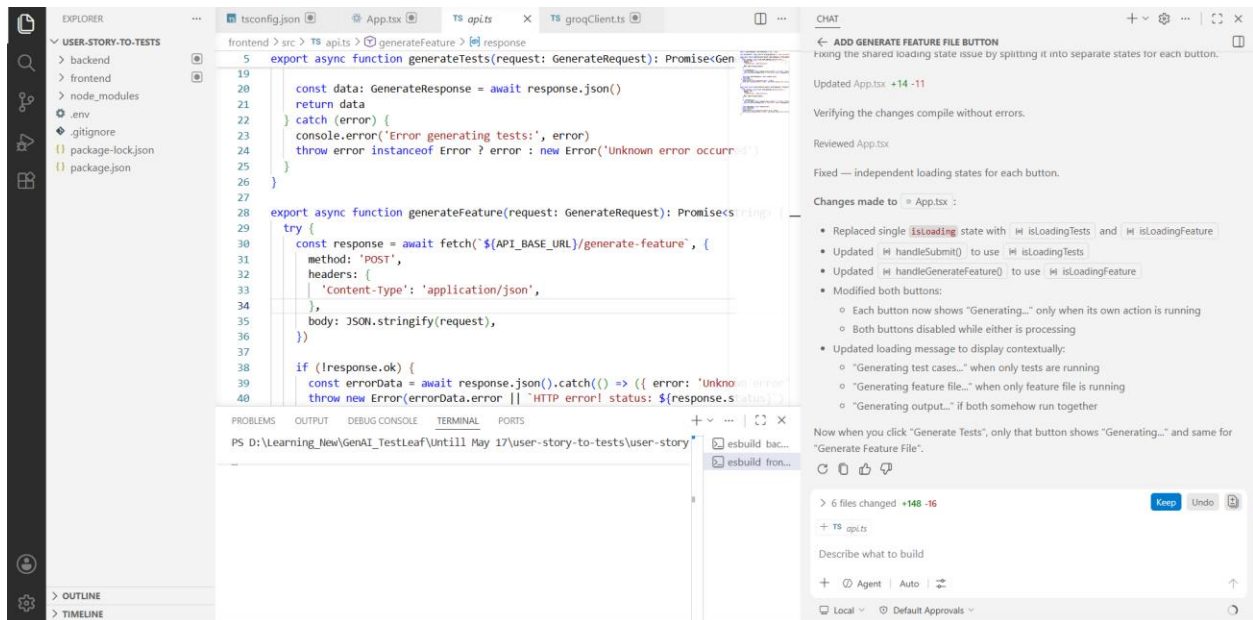

Step 4:

Prompt 3: Generate relevant **Scenario Outline** sections for the feature file



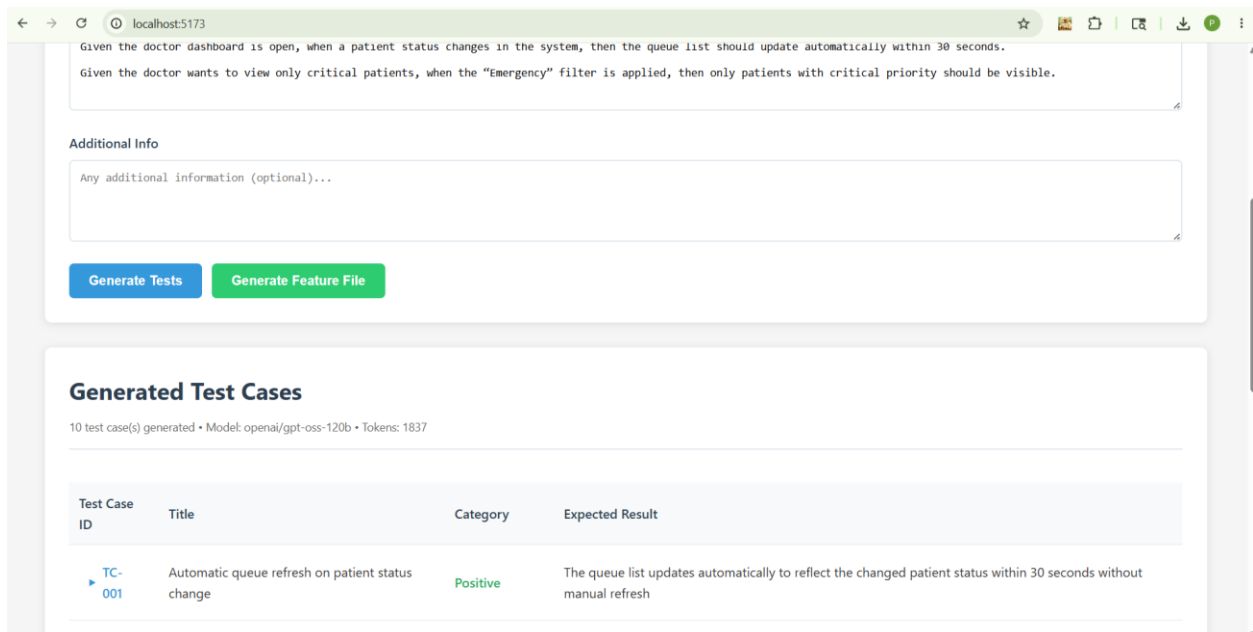
Prompt 4: when clicked on the one button both the buttons are getting clicked but the the functionalities are working fine When a particular button is clicked "Generating....." should come only for button which is clicked not both the buttons "Generate Testcases" and "Generate Feature file".





Step 5: Output:

Generate Tests:



Generate Feature File:

Given the doctor dashboard is open, when a patient status changes in the system, then the queue list should update automatically within 30 seconds.

Given the doctor wants to view only critical patients, when the "Emergency" filter is applied, then only patients with critical priority should be visible.

Additional Info

Any additional information (optional)...

Generate Tests

Generate Feature File

Generated Feature File

Download .feature File

Feature: Dynamic Queue List of AHC

As a ward doctor,
I want the queue list on the doctor dashboard to refresh automatically every 30 seconds
and to be filterable by patient priority (Emergency vs Routine),
so that I can attend to high-priority patients first.

```
Then the queue list does not update within 30 seconds
And a refresh timeout warning is displayed

# Happy path - emergency filter
Scenario: Applying the Emergency filter shows only critical patients
    Given the queue contains patients with priorities "Critical", "Routine", "High", "Low"
    When the doctor applies the "Emergency" filter
    Then only patients with priority "Critical" are visible
    And patients with priority "Routine", "High", or "Low" are hidden

# Main error flow - removing filter
Scenario: Removing the Emergency filter restores the full patient list
    Given the "Emergency" filter is applied
    When the doctor clears the filter
    Then all patients regardless of priority are visible

# Data-driven: various status changes
Scenario Outline: Automatic queue refresh for different patient status changes
    When a patient status changes to "<status>"
    Then the queue list updates within <expectedTime> seconds
    And the patient appears in the queue with status "<status>"
    Examples:
    | status | expectedTime |
    | Admitted | 30 |
    | Transferred | 30 |
    | Cancelled | 30 |
    | Pending | 30 |

# Data-driven: invalid status handling
Scenario Outline: Queue behavior when an invalid patient status is received
    When a patient status changes to "<invalidStatus>"
    Then the queue list does not update within 30 seconds
    And an error message "Invalid patient status: <invalidStatus>" is displayed
    Examples:
    | invalidStatus |
```

Feature File with Scenario Outline:

```
Feature: Dynamic Queue List of AHC
    As a ward doctor,
    I want the queue list on the doctor dashboard to refresh automatically every 30 seconds
    and to be filterable by patient priority (Emergency vs Routine),
    so that I can attend to high-priority patients first.
    Background:
        Given the doctor dashboard is open
    # Happy path - automatic refresh
    Scenario: Queue list updates automatically when a patient status changes
        When a patient status changes to "Admitted"
        Then the queue list updates within 30 seconds
        And the patient appears in the queue with status "Admitted"
    # Error flow - refresh timeout
    Scenario: Queue list does not update when system delay exceeds the refresh window
        When a patient status changes to "Discharged"
        And the system processing delay is 45 seconds
        Then the queue list does not update within 30 seconds
```



```

    And a refresh timeout warning is displayed
# Happy path - emergency filter
Scenario: Applying the Emergency filter shows only critical patients
    Given the queue contains patients with priorities "Critical", "Routine", "High", "Low"
    When the doctor applies the "Emergency" filter
    Then only patients with priority "Critical" are visible
    And patients with priority "Routine", "High", or "Low" are hidden
# Main error flow - removing filter
Scenario: Removing the Emergency filter restores the full patient list
    Given the "Emergency" filter is applied
    When the doctor clears the filter
    Then all patients regardless of priority are visible
# Data-driven: various status changes
Scenario Outline: Automatic queue refresh for different patient status changes
    When a patient status changes to "<status>"
    Then the queue list updates within <expectedTime> seconds
    And the patient appears in the queue with status "<status>"
    Examples:
        | status | expectedTime |
        | Admitted | 30 |
        | Transferred | 30 |
        | Cancelled | 30 |
        | Pending | 30 |
# Data-driven: invalid status handling
Scenario Outline: Queue behavior when an invalid patient status is received
    When a patient status changes to "<invalidStatus>"
    Then the queue list does not update within 30 seconds
    And an error message "Invalid patient status: <invalidStatus>" is displayed
    Examples:
        | invalidStatus |
        | Unknown |
        | "" |
        | null |
# Data-driven: Emergency filter with varying priority mixes
Scenario Outline: Emergency filter visibility with different priority sets
    Given the queue contains patients with priorities "<priorityList>"
    When the doctor applies the "Emergency" filter
    Then only patients with priority "<expectedVisible>" are visible
    And patients with other priorities are hidden
    Examples:
        | priorityList | expectedVisible |
        | Critical,Routine | Critical |
        | Critical,High,Low | Critical |
        | Routine,Low | (none) |
        | Critical | Critical |
        | (empty) | (none) |
# Data-driven: Refresh delay edge cases
Scenario Outline: Queue refresh behavior under different system processing delays
    When a patient status changes to "Admitted"
    And the system processing delay is <delay> seconds
    Then the queue list updates within <expectedOutcome> seconds
    Examples:
        | delay | expectedOutcome |
        | 10 | 30 seconds (successful) |
        | 30 | 30 seconds (successful) |
        | 31 | 30 seconds (successful) |
        | 45 | 30 seconds (timeout) |
# Data-driven: Filter removal with various initial states
Scenario Outline: Clearing the Emergency filter restores appropriate patient visibility
    Given the queue contains patients with priorities "<initialPriorities>"
    And the doctor applies the "Emergency" filter
    When the doctor clears the filter
    Then all patients with priorities "<initialPriorities>" are visible
    Examples:
        | initialPriorities |
        | Critical,Routine |
        | Critical,High,Low |
        | Routine,Low |
        | Critical |
        | (empty) |

```